SE301 – Project(2026-27)

Singapore Vertical Farming Research Management System (SVFRMS)

Advanced Programming and Design Project

Team Size: 3 Students per Team
Programming Language: Java
Project Type: Legacy Software Refactoring & Modernization

Table of Contents

1. Purpose 2
2. Project Overview 2
3. Existing System Description
   3.1. Crop Profile Management 3
   3.2. Environment Sensors 4
   3.3. Automated Devices 4
   3.4. Alert Management 5
   3.5. Report Generation 5
4. Project Requirements
   4.1. Architectural Refactoring 6
   4.2. SOLID Principles 6
   4.3. Design Pattern Integration 6
   4.4. Concurrency & Reliability 6
   4.5. Feature Extension 6
5. Deliverables 7
6. Instructions 8
7. Grading Rubric 8
8. Submission Deadline & Presentation 9
9. Use of AI Tools 9
10. Honor Code 9

1. Purpose

This project aims to expose students to real-world software maintenance and modernization challenges. Rather than building a system from scratch, Team will inherit a legacy Java application developed by a former intern.

Upon successful completion of this project, students should be able to:

1. Analyze and modernize a legacy Java application.
2. Apply clean code practices, SOLID principles, and appropriate design patterns.
3. Identify and resolve concurrency issues using multithreaded programming techniques.
4. Extend existing functionality, including crop management and alerting features, while maintaining software quality.
5. Produce professional technical documentation that supports future maintenance and development.

2. Project Overview

Singapore faces significant land constraints and relies on innovative agricultural technologies to maximize food production. To support this vision, a company has developed an indoor **Vertical Farming Research Facility** that uses vertically stacked growing chambers to cultivate crops such as tomatoes, strawberries, lettuce, cucumbers, chilies, and spinach in a carefully controlled environment.

Each crop requires unique environmental conditions, including temperature, humidity, light intensity, and nutrient levels. To maintain these conditions, the facility depends on a network of monitoring sensors and automated control devices that continuously collect data and regulate the growing environment.

Unfortunately, the software responsible for managing the facility was originally developed by a summer intern during the prototype phase. While the application still functions and supports day-to-day operations, years of growth have exposed serious software design issues. The codebase is a **single Java source file** has become difficult to understand, challenging to maintain, and increasingly risky to extend as new requirements emerge.

Your team has been hired as a group of elite senior engineers to assess the system before it can be deployed across additional farming facilities. Management is concerned that the current architecture will not scale, future enhancements will be expensive, and hidden defects could impact production reliability.

Your mission is to investigate, deconstruct, and modernize the legacy application. Acting as software consultants, your team must perform a comprehensive assessment of the existing system, identify its weaknesses, and transform it into a maintainable, extensible, and robust solution.

3. Existing System Description

The existing system provides integrated support for crop profile management, environmental monitoring, automated device control, alert generation, and report creation. It enables users to maintain crop-specific growth requirements, continuously monitor environmental conditions through sensors, automatically operate devices such as irrigation and lighting systems, receive alerts when conditions exceed predefined thresholds, and generate reports for analysing crop performance and overall system operations.

3.1 Crop Profile Management

The system shall allow users to create and manage crop profiles. Each crop profile stores the acceptable environmental operating range required for healthy crop growth, including the following attributes:

a) **Crop Name**: Name of the crop being monitored.
b) **Temperature Range**: Minimum and maximum temperature thresholds (°C).
c) **Humidity Range**: Minimum and maximum relative humidity thresholds (%).
d) **Soil Moisture Range**: Minimum and maximum soil moisture thresholds (%).
e) **Light Level Range**: Minimum and maximum light intensity thresholds (lux).

Examples:

Tomato

- Temperature: 22°C - 28°C
- Humidity: 60% - 80%
- Soil Moisture: 45% - 60%

Strawberry

- Temperature: 18°C - 24°C
- Humidity: 70% - 85%
- Soil Moisture: 50% - 65%

Lettuce

- Temperature: 15°C - 21°C
- Humidity: 65% - 85%
- Soil Moisture: 55% - 75%

3.2 Environment Sensors

The system shall collect and process environmental data from a set of sensors deployed within each growing chamber. These sensors provide real-time measurements that are used for monitoring conditions, generating alerts, and controlling automated devices. The supported sensor types include temperature, humidity, soil moisture, light intensity, and nutrient level sensors.

Example Sensor Reading:
Chamber 1: Temperature = 31°C, Humidity = 84%, Soil Moisture = 65%, Light Level = 750 lux, Nutrient Level = 72%.

3.3 Automated Devices

The system shall control a range of automated devices responsible for maintaining optimal growing conditions within each chamber. Device operations may be triggered automatically based on sensor readings, crop profile requirements, and user-defined control rules. The supported devices include:

1) **Water Pump**: Provides automated irrigation when soil moisture falls below the desired range.
2) **Cooling Fan**: Reduces temperature when excessive heat is detected.
3) **Heating Unit**: Maintains suitable temperatures during cold conditions.
4) **Grow Light**: Supplies artificial lighting when natural or measured light levels are insufficient.
5) **Nutrient Injector**: Delivers nutrients to support crop growth and health.
6) **Ventilation System**: Improves air circulation and helps regulate temperature and humidity.
7) **Shade Controller**: Reduces light exposure when conditions exceed crop-specific requirements.

Examples:

High Temperature
→ Cooling Fan Activated

Low Soil Moisture
→ Water Pump Activated

Low Nutrient Level
→ Nutrient Injector Activated

3.4 Alert Management

The system shall generate alerts whenever environmental conditions, nutrient levels, or sensor operations deviate from their expected ranges. Alerts help users identify issues early and take corrective actions to maintain optimal growing conditions. The supported alert types include:

1) **Temperature Alert**: Triggered when the temperature is above or below the defined threshold range.
2) **Humidity Alert**: Triggered when the humidity level is above or below the defined threshold range.
3) **Low Soil Moisture Alert**: Triggered when soil moisture falls below the minimum required level.
4) **Nutrient Deficiency Alert**: Triggered when nutrient levels are insufficient for healthy crop growth.
5) **Sensor Failure Alert**: Triggered when a sensor becomes unresponsive or reports invalid readings.

Alert Severity Levels:

- **INFO**: Informational notification that does not require immediate action.
- **WARNING**: Indicates a condition that may require attention to prevent potential issues.
- **CRITICAL**: Indicates a severe condition requiring immediate action to protect crops or system operations.

Example:

CRITICAL: Chamber 3 Temperature Exceeded Maximum Threshold.

3.5 Report Generation

The system shall generate operational and analytical reports to help users monitor facility performance, resource utilization, and crop health over time. These reports provide insights into environmental conditions, crop growth trends, and resource consumption patterns. The available reports include:

a. Daily Environmental Report
b. Weekly Crop Report
c. Water Usage Report
d. Energy Consumption Report
e. Nutrient Consumption Report

4. Project Requirements

Your team must analyze and improve the implementation provided. The goal is not to change system functionality. The goal is to improve the software design while preserving existing behavior.

4.1 Architectural Refactoring

The provided system consists primarily of a large monolithic Java class "SVFRMS".

Teams are required to analyze existing code structure and break it into a logical, multi-component system. A submission that remains in a single file will be disqualified.

4.2 Apply SOLID Principles

i. Determine whether classes perform multiple unrelated responsibilities.
ii. Determine whether new crop profiles require modification of existing code.
iii. Identify inheritance structures that violate expected behavior.
iv. Determine whether large interfaces should be decomposed.
v. Reduce direct dependencies on concrete implementations.

4.3 Design Pattern Integration

Teams can apply relevant design patterns learned in class and explain why they selected each pattern and what problem it solves.

4.4 Concurrency & Reliability

The intern introduced multithreading to improve performance. Unfortunately, several concurrency issues exist. Expecting race conditions, synchronization issue and deadlocks must be identified and rectified.

4.5 Feature Extension

The system should be designed to support the addition of new crops and the removal of existing crops with minimal modifications to the architecture.

Examples of additional crops:

1. Kale
2. Bell Pepper
3. Bok Choy
4. Eggplant.

The design should allow for future crop changes without requiring significant changes to the existing code.

5. Deliverables

Source Code Submission

Teams must submit a single **.zip** archive containing the following:

1. **src/** - The complete source code of the refactored Vertical Farming Management System organized using a Maven project structure.
2. **run.jar** - A single executable JAR file containing the application and all required dependencies. The application must be executable from the command line without requiring additional libraries or IDE configuration (You may use the provided pom.xml file to build the jar file. Upload the jar with all the dependencies included).
3. **README.pdf** - A technical report describing the identified issues, design improvements, concurrency fixes, applied design patterns, and any extensions implemented.

    1) **Report Length**: The report must be 3 - 6 pages, excluding the cover page.
    2) **Cover Page:** Include a cover page showing the group number and the names of all team members.
    3) **Legacy System Analysis:** Explain the purpose of the existing system and describe its main components.
    4) **Code Analysis and Limitations**: Identify the key design issues, limitations, or areas for improvement in the existing implementation. Support your observations with relevant examples from the legacy code.
    5) **Proposed Design:** Explain your proposed design and the key changes made to address the identified limitations of the legacy system. Justify your major design decisions.
    6) **Final Architecture:** Provide an explanation of your new system design. A high-level architectural/component diagram showing the main components and their interactions is required. Use relevant UML diagrams where appropriate, such as component diagrams and high-level sequence or collaboration diagrams.
    7) **Extensibility and Maintainability:** Explain how your proposed design supports future changes, such as adding new features or components, modifying existing functionality, or removing features with minimal impact on the rest of the system.
    8) **Design Patterns:** Identify and justify any relevant design patterns used in your proposed design. Explain why each pattern was selected and what problem it solves. If no design pattern is appropriate, explain your design choice.

Presentation

Each team shall deliver a presentation + demonstration:

1) Analysis of the Original System and Identified Issues.
2) Application of SOLID Principles and Refactoring Improvements.
3) Implementation and Demonstration of a New Crop Profile.
4) Identification and Resolution of Concurrency Issues (Race Conditions, Synchronization Problems, and Deadlocks).
5) Evidence of Improved Maintainability, Extensibility, and System Reliability.
6) Each team will be given a minimum of 5 Minutes for presentation and 2 minutes for the demonstration = Total 7 Minutes per team.

Teams may use **PowerPoint slides (recommended)** or another suitable presentation medium. The presentation should summarize the key findings and improvements made to the system and may reference supporting details contained in the submitted **README.pdf**.

6. Project Submission Instructions

1. **Group Formation:** Go to **eLearn** → **Class Info** → **Groups** to view the available groups.
2. **Self-Enrolment:** Each group can have a maximum of **3 students**. Please form your own group and complete the self-enrolment by **midnight tonight**.
3. **Students Without a Group:** If you are unable to find a group or face any difficulty, please let the TA or me know. We will try to help you find or form a group.
4. **Assignment Submission:** The Project details has been released under the **Assignments** section on eLearn. Please submit your work before the stated deadline. **Only one submission per group is required.** Include both your source code and report.pdf in the submission. Do not submit individually.

Development Environment

- Java 25 or later
- Maven permitted

Restrictions

Teams are expected to enhance the existing application while preserving its current behaviour. Removing existing features, replacing the system with a completely new implementation, or disregarding identified concurrency, design, and maintainability issues is not permitted.

7. Grading Rubric

Category	Description	Weight
Legacy System Analysis	Identification of design flaws, code smells, architectural weaknesses	10%
SOLID Principles	Quality of SOLID application and architectural improvements	20%
Refactoring Quality	Maintainability, modularity, readability and extensibility improvements	20%
Design Pattern Usage	Appropriate use and justification of design patterns	20%
Concurrency Analysis	Identification and explanation of race conditions and deadlocks	10%
Concurrency Improvements	Quality of synchronization and thread-safety solutions	10%

Presentation & Demonstration	Ability to explain design decisions and implementation	10%

Total = 100%

No extensions will be given for the submission.
Late submissions will be subjected to:
<=1 hour late - 50% penalty of final mark.
> 1 hour late - 100 % penalty (team will be awarded 0 for the project).

8. Submission Deadline & Presentation

Submission Deadline: Teams must submit their final project deliverables no later than **1 November 2026, 11:59 PM (Midnight)**. Late submissions may be subject to penalties in accordance with the module policy.

Presentation: Project presentations will be conducted on **2 November 2026 (Week 12)**. All team members are expected to participate and be prepared to explain their design decisions, refactoring approach, and implemented improvements.

9. Use of Generative AI

You are allowed to use Large Language Models (LLMs) such as ChatGPT, Google AI Studio or GitHub Copilot to assist with your project work. However, it is your responsibility to fully understand any code or concepts generated using these tools.

You may be asked to explain your code and design decisions during reviews or presentations. Failure to demonstrate understanding will impact your grade. Use the AI tools to learn and accelerate your development - not as a shortcut.

10. Honor Code

SMU expects students to abide by the honor code and The SMU Student Code of Conduct. Most importantly, your answers must be your own! So, do not discuss these specific problems with other teams, seniors etc. Remember, plagiarism and conferring with your peers is academic dishonesty and carries severe consequences.